Real-Time Volumetric Cloud Rendering

github.com/cyberfrank

S0007E

Project Goal

- Implement a volumetric cloud rendering system for real-time games that run within a **respectable millisecond budget** (≤2ms).
- Cloud visuals should be **acceptable** and **moderately convincing** compared to real-life.

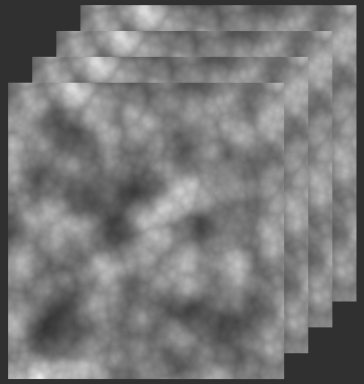




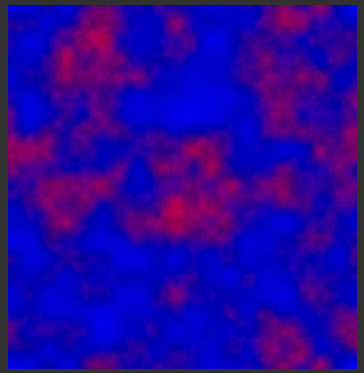
Rockstar Advanced Game Engine
Red Dead Redemption 2

Cloud Modeling

- Combinations of 3D noise at various frequencies creates the cloud foundation
  - Computed once by a separate command-line tool
- A *weather texture* provides the artistic control
  - Hand drawn or generated by a shader program
  - Contains *cloud coverage* and *cloud type* (with room for more!)
- Model only focuses on lower level cloud layer



volume noise texture



weather map texture



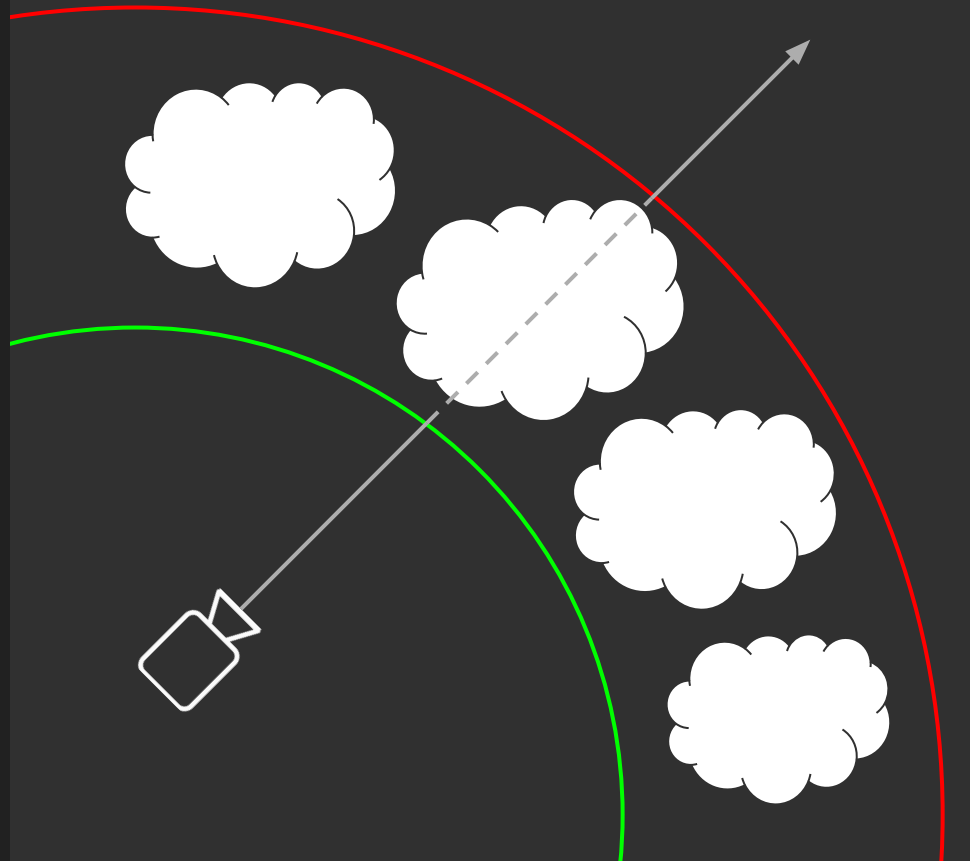
Without using noise as
base probability for cloud density



With noise textures
(low frequency + high frequency)

Volume Ray Marching

- In a volumetric ray march you use 3D textures to…
  - Build up an alpha channel
  - Calculate lighting
  - Compose the final pixel color
- A *bounding volume* determines where the ray marching may take place
  - Planetary scale solution that only captures samples between an inner and outer sphere
  - Allows for clouds to naturally bend over the horizon
- More ray march steps yields better visual quality!

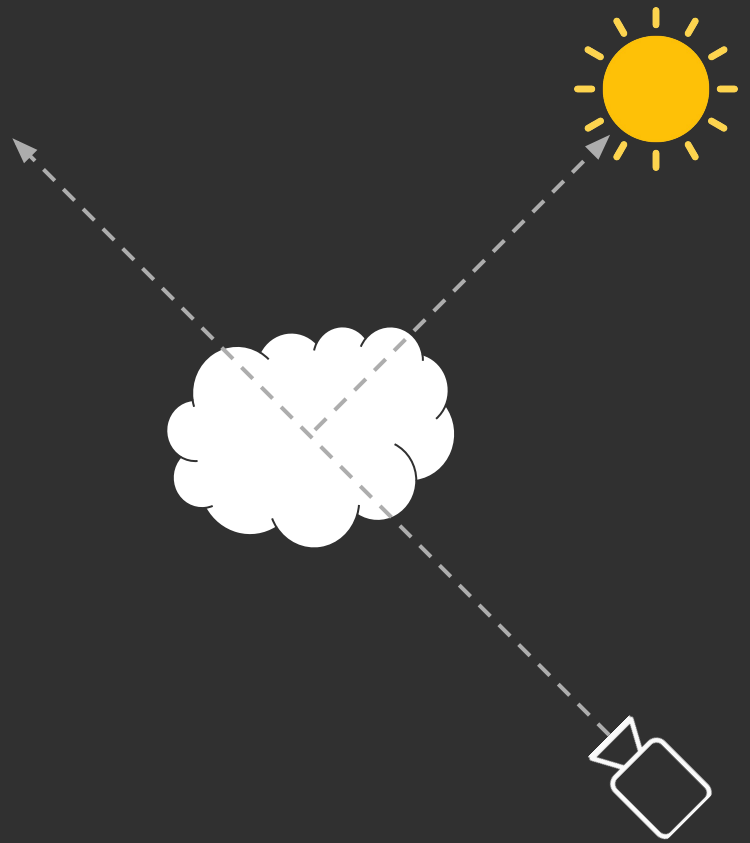


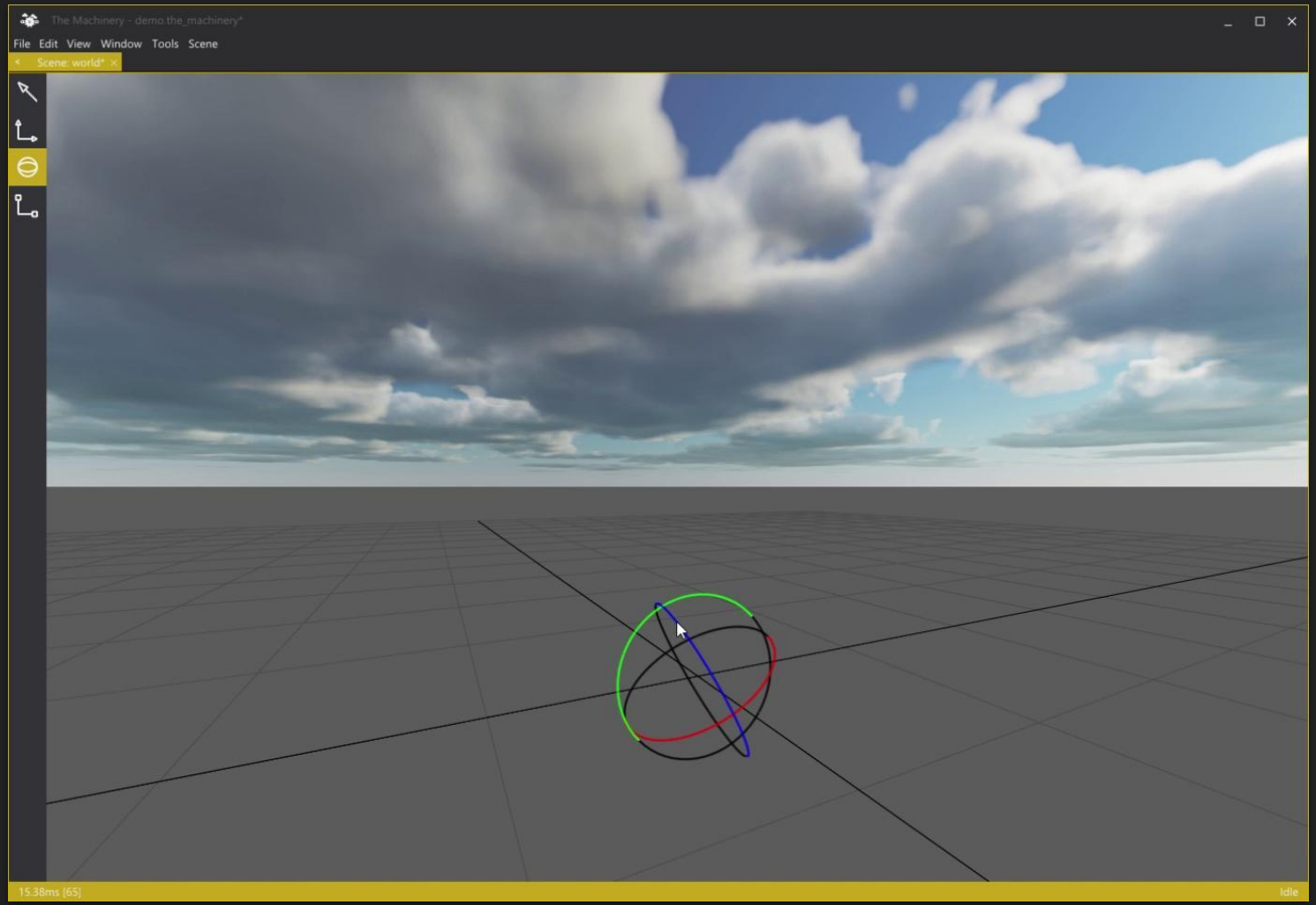


1 step 2 steps 4 steps 16 steps 64 steps

Lighting

- Lighting evaluated for each sample that yield a density larger than zero
  - Physically based with some simplifications and artistic liberties
- Additional samples must be captured in the sun direction for each ray step
  - Allows for self and inter-cloud shadowing
  - Required to convey a sense of depth
  - Expensive!



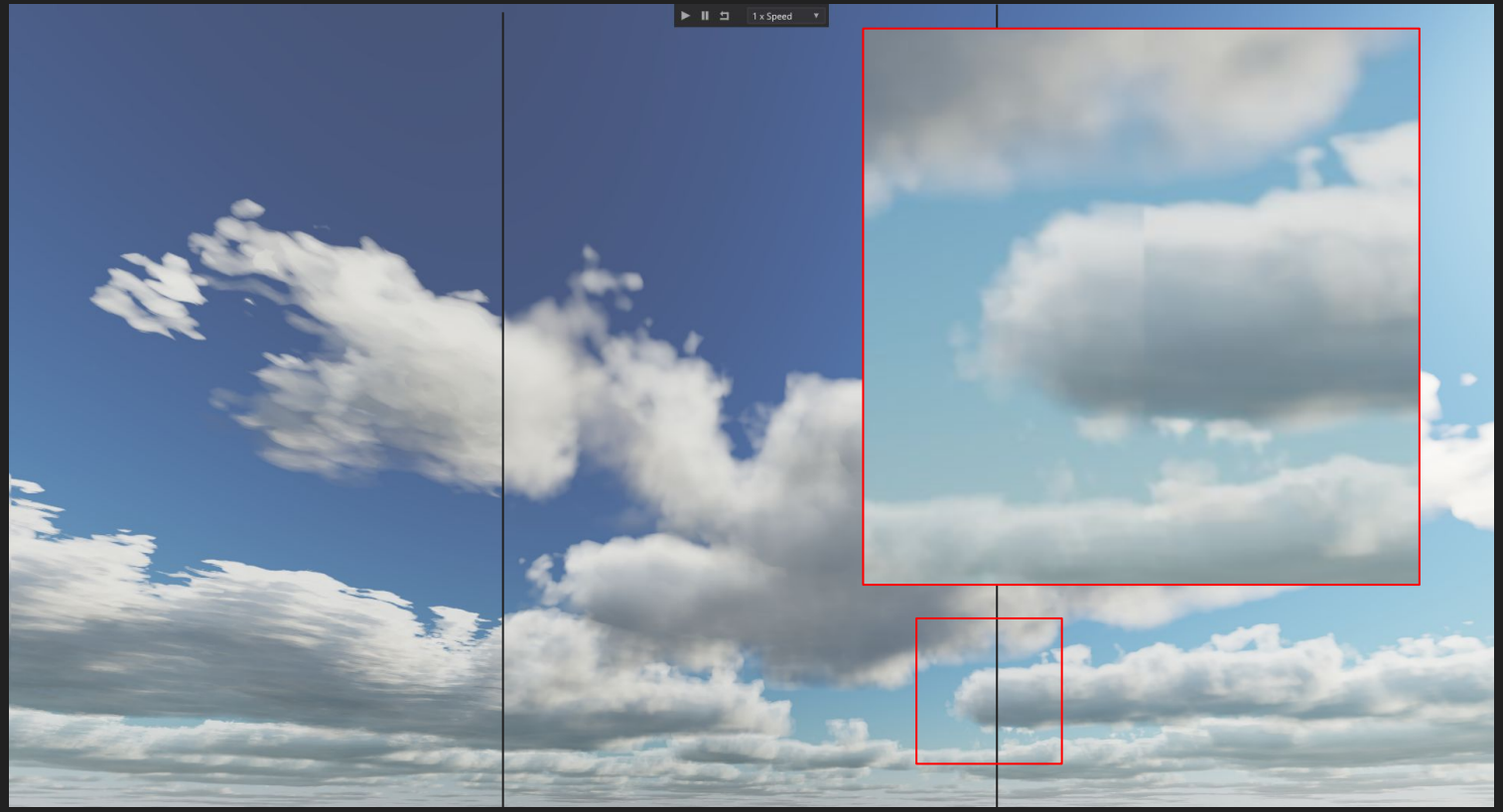

The Machinery - demo.the_machinery*
File Edit View Window Tools Scene
Scene: world*
15.38ms [65]
Idle

Optimization 1

- Too many samples used to gain an acceptable visual result. Additional techniques required for ≤2ms!

- **Solution:**
  - Temporal integration of the image achieved using an exponential moving average
  - For each frame, offset the ray origin by random value in range [0,1]
  - Blend current frame with *previous frame* according to a constant blend factor
  - Must sample previous frame by re-projected position to avoid ghosting





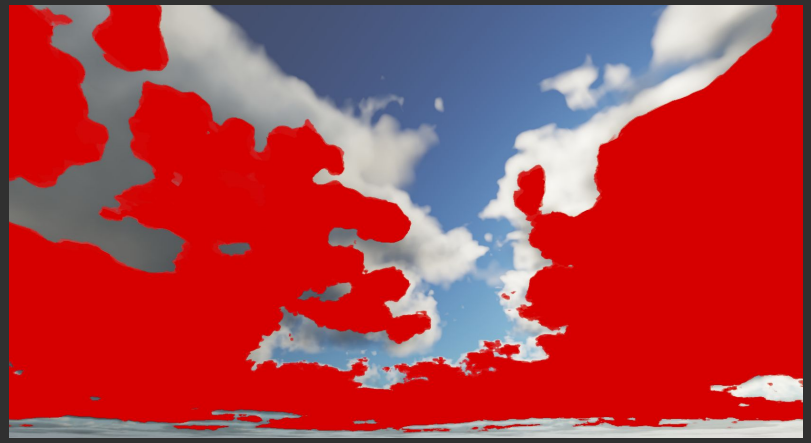
no temporal integration
13 samples

temporal integration
13 samples

no temporal integration
256 samples (reference)

Optimization 2

- Good idea to opt out of ray-marching as soon as possible to avoid O(n*m) worst case per pixel.
- **Solution:**
  - Exit when accumulated alpha along ray was too low (cloud very much visible in this direction, additional samples would be trivial)
  - Or… if the previous frame accumulated alpha was too high, skip the ray march entirely and force the next frame to be rendered instead.





Optimization 3

- GPU may still struggle at ≥1080p, even when sample count is low.
- **Solution:**
  - Render clouds at half resolution!
  - Minimal visual quality loss
  - Even lower resolutions possible at the cost of a blurrier image
  - Clouds are in general already “blurry” by nature



half resolution



half resolution render target (≅1ms)

full resolution render target (≫2ms)

Results

- Goal achieved!
- Run-time performance: ~1ms on target machine
- Additional optimizations still possible:
  - Minimize texture reads
  - Texture LODs
  - Optimize code path

